

Лекция: Транспортные протоколы и стандарты форматирования сообщений

1. Транспортный уровень в модели OSI

Транспортный уровень (4-й уровень модели OSI) отвечает за:

- надёжную передачу данных между узлами сети;
- контроль порядка доставки пакетов;
- управление ошибками и повторную отправку потерянных данных;
- регулирование потока данных;
- мультиплексирование и демultipлексирование (разделение потоков данных между приложениями с помощью портов).

2. Основные транспортные протоколы

TCP (Transmission Control Protocol) — ориентированный на соединение протокол с гарантированной доставкой:

- устанавливает соединение через «трёхстороннее рукопожатие» (SYN → SYN-ACK → ACK);
- разбивает данные на сегменты с номерами последовательности;
- подтверждает получение каждого сегмента;
- регулирует поток данных через «скользящее окно»;
- гарантирует доставку в правильном порядке без потерь и дубликатов.

Применение TCP: веб-серфинг (HTTP/HTTPS), электронная почта (SMTP, POP3, IMAP), передача файлов (FTP).

UDP (User Datagram Protocol) — упрощённый протокол без установления соединения:

- не гарантирует доставку пакетов;
- не контролирует порядок получения;
- минимальные задержки за счёт отсутствия подтверждений;
- подходит для быстрой передачи небольших объёмов данных.

Применение UDP: стриминг видео/аудио, VoIP (Skype, Zoom), онлайн-игры, DNS-запросы.

Другие транспортные протоколы:

- **SCTP (Stream Control Transmission Protocol)** — сочетает надёжность TCP с многопоточностью и поддержкой нескольких IP-адресов (используется в телекоммуникациях).

- **DCCP (Datagram Congestion Control Protocol)** — обеспечивает контроль перегрузки для потоковых приложений без гарантии доставки.
- **RUDP (Reliable User Datagram Protocol)** — расширяет UDP механизмами подтверждения доставки и повторной отправки.
- **QUIC** — современный протокол Google, сочетающий надёжность TCP с низкой задержкой UDP (используется в HTTP/3).
- **RTP (Real-time Transport Protocol)** — предназначен для передачи аудио/видео в реальном времени (часто работает с RTCP для контроля качества).

3. Стандарты форматирования сообщений

На транспортном уровне данные передаются в виде сегментов (TCP) или дейтаграмм (UDP). Их структура включает:

Заголовок TCP:

- порты отправителя и получателя (16 бит каждый);
- номер последовательности (32 бита) — определяет позицию данных в потоке;
- номер подтверждения (32 бита) — указывает, какие данные получены;
- длина заголовка (4 бита);
- флаги (6 бит): SYN, ACK, FIN, RST и др.;
- размер окна (16 бит) — объём данных, который может принять получатель;
- контрольная сумма (16 бит);
- указатель срочности (16 бит).

Заголовок UDP:

- порт отправителя (16 бит);
- порт получателя (16 бит);
- длина дейтаграммы (16 бит);
- контрольная сумма (16 бит).

Форматы прикладного уровня (для передачи сообщений):

- **JSON (JavaScript Object Notation)** — лёгкий формат обмена данными, читаемый человеком и машиной. Пример:
- {
- "name": "Alice",
- "age": 30,

- "city": "Moscow"
- }
- **XML (eXtensible Markup Language)** — структурированный формат с тегами. Пример:
- <person>
- <name>Alice</name>
- <age>30</age>
- <city>Moscow</city>
- </person>
- **Protocol Buffers (Protobuf)** — бинарный формат от Google, компактный и быстрый. Описывается схемой:
- message Person {
- string name = 1;
- int32 age = 2;
- string city = 3;
- }
- **MessagePack** — бинарная альтернатива JSON, более компактная.
- **CSV (Comma-Separated Values)** — табличный формат для данных с разделителями. Пример:
- name,age,city
- Alice,30,Moscow

4. Взаимодействие протоколов и форматов

1. **Прикладной уровень** формирует сообщение в формате JSON/XML/Protobuf.
2. **Транспортный уровень** разбивает его на сегменты (TCP) или упаковывает в дейтаграммы (UDP), добавляя заголовки.
3. **Сетевой уровень** инкапсулирует сегменты в IP-пакеты с адресами отправителя/получателя.
4. На приёмной стороне процесс идёт в обратном порядке: пакеты → сегменты → сборка данных → преобразование в исходный формат.

5. Выбор протокола и формата

Критерий TCP

UDP

Критерий	TCP	UDP
Надёжность	Высокая (подтверждения, повторные отправки)	Низкая (без гарантий)
Задержка	Выше (из-за управления соединением)	Минимальная
Скорость	Зависит от канала	Максимальная
Применение	Веб, почта, файлы	Стриминг, игры, VoIP

Выбор формата:

- JSON/XML — для человекочитаемых API и веб-сервисов;
- Protobuf/MessagePack — для высоконагруженных систем, где важна скорость и компактность;
- CSV — для табличных данных и экспорта.

Заключение

Транспортные протоколы (TCP, UDP и др.) обеспечивают доставку данных, а форматы (JSON, XML, Protobuf) — их структурирование. Правильный выбор комбинации зависит от задачи: критична ли надёжность, допустимы ли потери, важна ли скорость. Понимание этих принципов позволяет проектировать эффективные сетевые приложения.

Хотите, я раскрою какой-либо раздел подробнее?